



**ALLIANCE
UNIVERSITY**
Private University established in Karnataka State by Act No. 84 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY

Bachelor of Computer Applications

Semester – V

Deep learning

Assignment No.: 9

Title: Optimizer Comparison Using a Neural Network

Submitted By

Name: Dipin Roka

Reg. No.:2411021240007

Date:7/10/2026

[Github repo](#)

Submitted To:

Faculty Name: Nagasri p

Department of Computer Application

Alliance University

Chandapura - Anekal Main Road, Anekal Bengaluru – 562106

```
""" Project 9: Optimizer Comparison Using a Neural
Network
Dataset: Iris Classification Dataset
Optimizers compared: SGD vs Adam
```

```
Task 1: Build a simple neural network for Iris classification
Task 2: Train the model using SGD and Adam optimizers
Task 3: Compare training results and classification performance
"""
```

```
import os import random import numpy as np import
torch import torch.nn as nn from torch.utils.data
import TensorDataset, DataLoader from
sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split from sklearn.preprocessing
import StandardScaler from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix import matplotlib matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ----- Setup -----SEED
= 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

RESULTS_DIR = os.path.join(os.path.dirname(__file__), "results")
os.makedirs(RESULTS_DIR, exist_ok=True)

DEVICE = torch.device("cpu")
print(f"Using device: {DEVICE}")

# Hyperparameters (kept identical for fair comparison except optimizer)
HIDDEN1 = 16
HIDDEN2 = 16
EPOCHS = 100
BATCH_SIZE = 16
LR = 0.01
SGD_MOMENTUM = 0.9
TEST_SIZE = 0.2

# ----- Task 1: Data + Model -----
print("\n===== Task 1: Build Simple Neural Network on Iris Dataset =====")
iris = load_iris()
X, y = iris.data, iris.target print(f"Dataset: Iris | Samples: {X.shape[0]} |
Features: {X.shape[1]} | Classes:
{iris.target_names.tolist()}")
print(f"Feature names: {iris.feature_names}")

# Train/test split (stratified to preserve class balance)
X_train, X_test, y_train, y_test = train_test_split( X, y,
test_size=TEST_SIZE, random_state=SEED, stratify=y
)
print(f"Train size: {X_train.shape[0]}, Test size: {X_test.shape[0]}")

# Feature scaling (important for neural networks + SGD)
```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# PyTorch datasets / loaders
train_ds = TensorDataset(torch.tensor(X_train, dtype=torch.float32),
                          torch.tensor(y_train, dtype=torch.long))
test_ds = TensorDataset(torch.tensor(X_test, dtype=torch.float32),
                        torch.tensor(y_test, dtype=torch.long))
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE)

class IrisMLP(nn.Module):
    """Simple feedforward network: 4 -> 16 -> 16 -> 3"""
    def __init__(self, input_dim=4, hidden1=HIDDEN1, hidden2=HIDDEN2, num_classes=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden1),
            nn.ReLU(),
            nn.Linear(hidden1, hidden2),
            nn.ReLU(),
            nn.Linear(hidden2, num_classes)
        )

    def forward(self, x):
        return self.net(x)

print("\nModel architecture:")
print(IrisMLP())

# ----- Task 2: Training function -----
def train_model(optimizer_name="adam"):
    print(f"\n===== Task 2: Training with {optimizer_name.upper()} =====")
    torch.manual_seed(SEED) # same init for fair comparison
    model = IrisMLP().to(DEVICE)
    criterion = nn.CrossEntropyLoss()

    if optimizer_name.lower() == "sgd":
        optimizer = torch.optim.SGD(model.parameters(), lr=LR, momentum=SGD_MOMENTUM)
    elif optimizer_name.lower() == "adam":
        optimizer = torch.optim.Adam(model.parameters(), lr=LR)
    else:
        raise ValueError("optimizer_name must be 'sgd' or 'adam'")

    train_losses, train_accs, test_accs = [], [], []

    for epoch in range(1, EPOCHS + 1):
        model.train()
        epoch_loss = 0.0
        correct, total = 0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)

```

```

optimizer.zero_grad()
out = model(xb) loss =
criterion(out, yb)
loss.backward()
optimizer.step()
epoch_loss += loss.item() * xb.size(0)
_, pred = out.max(1)
correct += (pred == yb).sum().item()
total += yb.size(0)

train_loss = epoch_loss / total
train_acc = correct / total

# Evaluate on test set each
epoch model.eval() t_correct,
t_total = 0, 0 with
torch.no_grad():
    for xb, yb in test_loader: xb, yb =
        xb.to(DEVICE), yb.to(DEVICE)
        _, pred = model(xb).max(1) t_correct
        += (pred == yb).sum().item() t_total
        += yb.size(0)
test_acc = t_correct / t_total

train_losses.append(train_loss)
train_accs.append(train_acc)
test_accs.append(test_acc)

if epoch == 1 or epoch % 10 == 0:
    print(f"Epoch {epoch:3d}/{EPOCHS} | Loss: {train_loss:.4f} | "
          f"Train Acc: {train_acc:.4f} | Test Acc: {test_acc:.4f}")

# Final evaluation
model.eval() all_pred,
all_true = [], [] with
torch.no_grad():
    for xb, yb in test_loader: xb =
        xb.to(DEVICE) _, pred =
        model(xb).max(1)
        all_pred.extend(pred.cpu().numpy())
        all_true.extend(yb.numpy())

final_acc = accuracy_score(all_true, all_pred)
print(f"\n[{optimizer_name.upper()}] Final Test Accuracy: {final_acc:.4f}")
print(classification_report(all_true, all_pred, target_names=iris.target_names))
print(f"Confusion Matrix:\n{confusion_matrix(all_true, all_pred)}")

return {
    "model": model,
    "losses": train_losses,
    "train_accs": train_accs,
    "test_accs": test_accs,
    "final_acc": final_acc,

```

```

        "y_true": all_true,
        "y_pred": all_pred,
    }

sgd_results = train_model("sgd")
adam_results = train_model("adam")

# ----- Task 3: Comparison -----
print("\n===== Task 3: Compare SGD vs Adam =====")
print(f"SGD  final train loss: {sgd_results['losses'][-1]:.4f} | "
      f"train acc: {sgd_results['train_accs'][-1]:.4f} | test acc: "
      f"{sgd_results['final_acc']:.4f}")
print(f"Adam final train loss: {adam_results['losses'][-1]:.4f} | "
      f"train acc: {adam_results['train_accs'][-1]:.4f} | test acc: "
      f"{adam_results['final_acc']:.4f}")

# Epoch to reach 90% train accuracy (convergence speed)
def epochs_to_target(acc_list, target=0.90):
    for i, a in enumerate(acc_list, 1):
        if a >= target:
            return i
    return None

print(f"Epochs to reach 90% train acc -> SGD: "
      f"{epochs_to_target(sgd_results['train_accs'])}, " f"Adam: "
      f"{epochs_to_target(adam_results['train_accs'])}")
print(f"Epochs to reach 90% test acc -> SGD: "
      f"{epochs_to_target(sgd_results['test_accs'])}, " f"Adam: "
      f"{epochs_to_target(adam_results['test_accs'])}")

# --- Plot 1: Training loss ---plt.figure(figsize=(8,
5)) plt.plot(sgd_results["losses"], label="SGD")
plt.plot(adam_results["losses"], label="Adam")
plt.title("Training Loss: SGD vs Adam (Iris MLP)")
plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, "loss_comparison.png"))
print("Saved: results/loss_comparison.png")

# --- Plot 2: Test accuracy ---plt.figure(figsize=(8,
5))
plt.plot(sgd_results["test_accs"], label="SGD (test)")
plt.plot(adam_results["test_accs"], label="Adam (test)")
plt.plot(sgd_results["train_accs"], "--", alpha=0.6, label="SGD (train)")
plt.plot(adam_results["train_accs"], "--", alpha=0.6, label="Adam (train)")
plt.title("Accuracy: SGD vs Adam (Iris MLP)") plt.xlabel("Epoch")
plt.ylabel("Accuracy") plt.ylim(0, 1.05) plt.legend() plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, "accuracy_comparison.png"))
print("Saved: results/accuracy_comparison.png")

```

```

# --- Plot 3: Confusion matrices side by side --from
sklearn.metrics import ConfusionMatrixDisplay fig,
axes = plt.subplots(1, 2, figsize=(10, 4))
ConfusionMatrixDisplay(confusion_matrix(sgd_results["y_true"],
                                       sgd_results["y_pred"]),
                       display_labels=iris.target_names).plot(ax=axes[0],
                                                             colorbar=False)

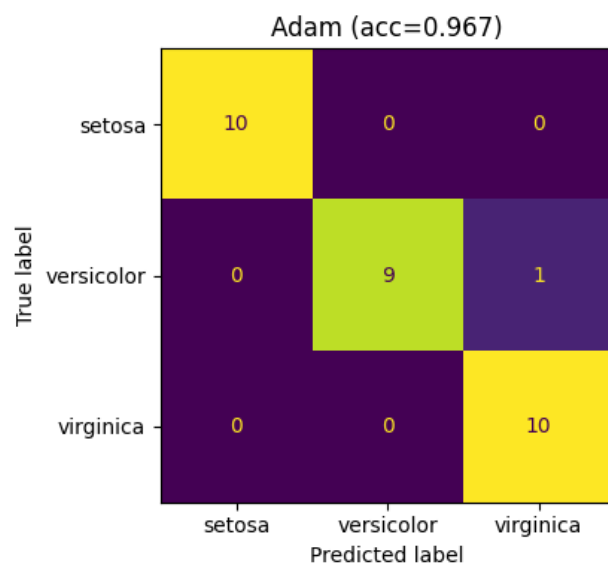
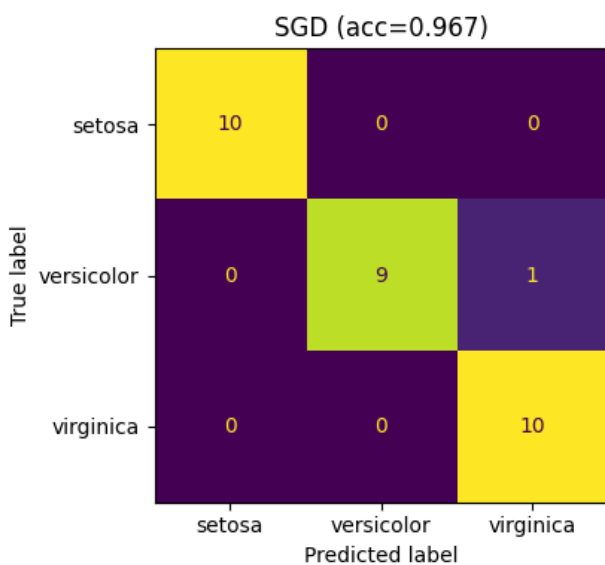
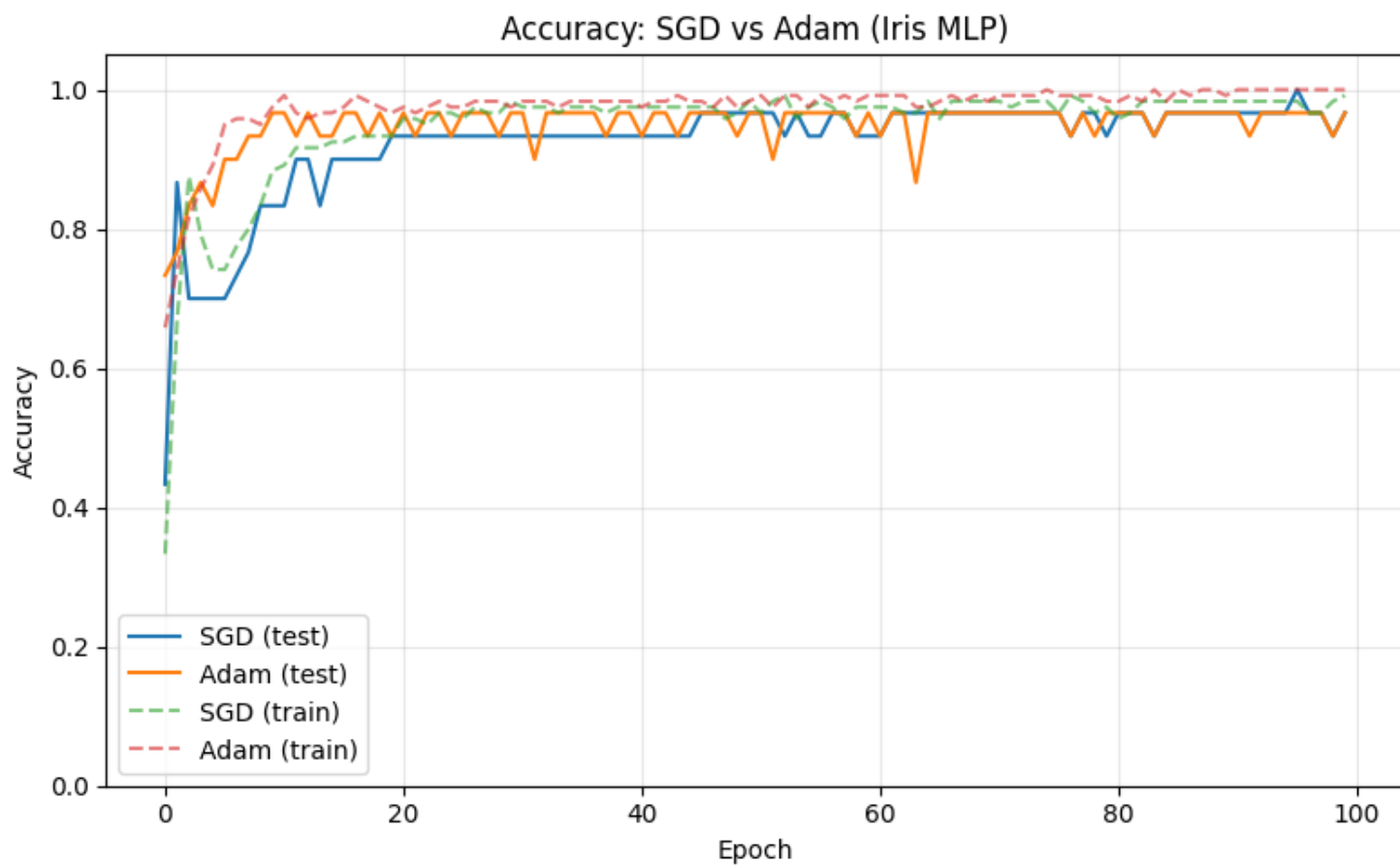
axes[0].set_title(f"SGD (acc={sgd_results['final_acc']:.3f})")
ConfusionMatrixDisplay(confusion_matrix(adam_results["y_true"],
                                       adam_results["y_pred"]),
                       display_labels=iris.target_names).plot(ax=axes[1],
                                                             colorbar=False)

axes[1].set_title(f"Adam (acc={adam_results['final_acc']:.3f})")
plt.tight_layout()
plt.savefig(os.path.join(RESULTS_DIR, "confusion_matrices.png"))
print("Saved: results/confusion_matrices.png")

# --- Save metrics summary --with open(os.path.join(RESULTS_DIR,
"metrics_summary.txt"), "w") as f:
    f.write("Project 9: Optimizer Comparison (Iris MLP)\n")
    f.write("=" * 50 + "\n")
    f.write(f"Model: 4 -> {HIDDEN1} -> {HIDDEN2} -> 3 (ReLU, CrossEntropyLoss)\n")
    f.write(f"Epochs={EPOCHS}, Batch={BATCH_SIZE}, LR={LR}, SGD
momentum={SGD_MOMENTUM}\n")
    f.write(f"Split: 80/20 stratified, StandardScaler, seed={SEED}\n\n")
    f.write(f"SGD | final loss={sgd_results['losses'][-1]:.4f} | "
           f"train acc={sgd_results['train_accs'][-1]:.4f} | test
acc={sgd_results['final_acc']:.4f}\n")
    f.write(f"Adam | final loss={adam_results['losses'][-1]:.4f} | "
           f"train acc={adam_results['train_accs'][-1]:.4f} | test
acc={adam_results['final_acc']:.4f}\n\n")
    f.write(f"Epochs to 90% train acc:
           SGD={epochs_to_target(sgd_results['train_accs'])} "
           f"Adam={epochs_to_target(adam_results['train_accs'])}\n")
    f.write(f"Epochs to 90% test acc:
           SGD={epochs_to_target(sgd_results['test_accs'])} "
           f"Adam={epochs_to_target(adam_results['test_accs'])}\n")
print("Saved: results/metrics_summary.txt") print("\nDone! Check
the 'results' folder for plots and metrics.")

```

Output



Training Loss: SGD vs Adam (Iris MLP)

